

An Adaptive Large Neighborhood Search for the Prize-Collecting Skill Vehicle Routing Problem

Umais Chaudhary, Rehman Kerimov, Dustin Schulz, Marco Leander vom Orde
31-MM34 Metaheuristics, SoSe 26, Bielefeld University

July 2026

1 Introduction

A pharmacy delivers medication to patients at home. Each delivery may require a qualification (administering an injection, handling refrigerated goods), each courier holds a subset of these skills, and not every patient can be served within the couriers’ shifts. The dispatcher therefore faces a *prize-collecting* problem: select which customers to serve at all, and route the couriers so that the collected profit is maximized.

Formally, an instance consists of customers $C = \{1, \dots, n\}$ with profit p_i , service time s_i , time window $[r_i, d_i]$ and required skill set Q_i , and couriers $B = \{1, \dots, b\}$ with skill set S_k and shift $[a_k, e_k]$. All tours start and end at the pharmacy (node 0); travel times are Euclidean distances rounded down. Courier k may serve customer i only if $Q_i \subseteq S_k$; service must start within $[r_i, d_i]$ (early arrival waits), and the courier must return to the depot by e_k . Each customer is served at most once; unserved customers simply yield no profit. The objective is to maximize $\sum_{i \in C'} p_i$ over the set C' of served customers. The empty plan is always feasible, so the problem is one of profitable selection rather than mere feasibility.

As a yardstick we use the *singleton upper bound* $UB = \sum_i p_i \cdot [\exists k : i \text{ alone is feasible for } k]$: the profit of all customers that at least one courier could serve in a dedicated round trip. UB ignores that customers compete for shift time, so it is loose—particularly when few couriers share many reachable customers—but it is cheap, instance-independent and allows relative comparisons across instance sizes.

The report follows our development path: the literature fixes the method family (Section 2); a simple construction heuristic fixes the baseline (Section 3.1); engineering makes move evaluation cheap enough for pure Python under a wall-clock budget (Section 3.5); operator breadth is then added and *measured*—by a leave-one-out ablation and a parameter study (Section 4)—rather than argued. Section 5 reports the resulting performance under the official 30 s budget and in 10-minute runs.

2 Choice of Method

2.1 Related Work

Selecting profitable customers under routing constraints is the *orienteering* / *prize-collecting* family: the team orienteering problem with time windows (TOPTW) is our problem without skills and shifts; surveys by Vansteenwegen et al. [5] and Gunawan et al. [6] show that the strongest TOPTW heuristics are insertion-based local search and large neighborhood schemes, e.g. the LNS of Labadie et al. [7] and the iterated approach of Hu and Lim [8]. Skill compatibility is studied as the Skill VRP by Cappanera et al. [9] and, closest to our setting, in service-technician routing by Kovacs et al. [10], who solve a prize-collecting routing problem with qualifications and time windows using adaptive large neighborhood search (ALNS). Home health care routing shares the same structure [11]. ALNS itself goes back to Shaw’s relatedness-based removal [1] and was formalized with adaptive operator weights and simulated-annealing acceptance by Ropke and Pisinger [2, 3]. Insertion feasibility in constant time via forward time slacks is due to Savelsbergh [4]. More recent LNS variants contribute additional operators: sequence- and saving-based removals [13], time-oriented relatedness [12], and slack-induced string removals [14].

Algorithm 1: ALNS for the prize-collecting Skill VRP

```
 $x \leftarrow$  greedy construction;  $x^* \leftarrow x$ ;  $T \leftarrow T_0$ 
while wall-clock time remains do
  choose destroy  $d$  and repair  $r$  by roulette wheel over weights  $w$ 
   $x' \leftarrow r(d(x))$  (remove  $q$  customers, reinsert removed + sampled unserved)
  if  $\exp((f(x') - f(x))/T) > u \sim U(0, 1)$  then  $x \leftarrow x'$  (penalized score  $f$ )
  if  $x'$  feasible and  $\text{profit}(x') > \text{profit}(x^*)$  then  $x^* \leftarrow x'$ 
  credit  $d$  and  $r$  with score  $\sigma \in \{25, 10, 3, 0\}$ ; record their runtime
  every  $L$  iterations:  $w \leftarrow (1 - \rho)w + \rho \frac{\bar{\sigma}}{1 + (\bar{t}/\tau)^\alpha}$  (runtime-normalized utility)
  update  $T$  on the exponential wall-clock schedule  $T_0 \rightarrow T_{\min}$ 
  if 500 consecutive rejections then  $x \leftarrow$  randomized greedy; reheat  $T$ 
return  $x^*$ 
```

Figure 1: Main loop. f = profit minus infeasibility penalties; $\bar{\sigma}, \bar{t}$ = mean score and runtime of an operator in the last segment; L = segment length, ρ = reaction factor, τ, α = runtime normalization constants.

2.2 ALNS justification

Three properties of the problem drove the choice. First, the search must jointly decide *selection* and *routing*: destroy/repair moves manipulate the served set directly (a repair step may insert previously unserved customers), whereas classical edge-exchange neighborhoods keep the customer set fixed and would need a separate selection mechanism. Second, the benchmark is extremely heterogeneous ($n = 50$ to 1000, 2 to 25 couriers, skill scarcity $k = 0$ to 2): with a fixed 30 s budget in pure Python our iteration throughput spans two orders of magnitude ($\sim 30,000$ iterations on $n=50$, ~ 600 on $n=1000$). An *adaptive* scheme that reweights operators online by their measured usefulness *and cost* transfers across this range without per-instance tuning. Third, skills and tight windows make feasibility rugged; simulated-annealing acceptance over a penalized objective lets the search cross infeasible ridges. ALNS combines all three ingredients in one well-understood framework [2] and was validated on the closest published relative of our problem [10].

3 Approach Description

3.1 Initial Greedy solution

The construction heuristic sorts all customers that at least one courier could serve alone by *decreasing profit* and inserts each at the cheapest feasible position (minimal travel-time increase) over all skill-compatible couriers; customers without a feasible position are left unserved. The skill prefilter (per-customer list of compatible couriers) is precomputed once. Construction takes < 0.5 s even for $n = 1000$ and reaches on average 46.4 % of UB (range 40.5–52.6 %) over the 20 benchmark instances.

To validate the profit ordering we compared against random insertion orders (same cheapest-insertion machinery, 5 seeds): random construction reaches only 68.6 % of the greedy profit on average (e.g. $n=1000$: $18,648 \pm 831$ vs. $27,323$), with the gap widening on larger instances. Profit ordering is essentially free and supplies about two thirds of the final solution quality (cf. Table 2); we also tried a multi-start variant over four orderings (profit, profit density, fewest compatible couriers, hybrid), which improves the *start* by up to 10 % but had no measurable effect on the *final* result—the ALNS absorbs start differences within seconds (Fig. 2)—so the submitted solver keeps the single profit ordering.

3.2 ALNS Framework

Algorithm 1 shows the main loop. Per iteration one destroy and one compatible repair operator are drawn by roulette-wheel selection over adaptive weights, applied to the current solution, and the result is accepted by simulated annealing on a penalized score. Weights are updated segment-wise; after a stagnation phase the search restarts from a randomized greedy solution with reheated temperature.

3.3 Destroy operators

Fifteen destroy operators fall into five groups: **random**: uniform removal; **cost-based**: worst profit density, worst detour (two variants: recompute vs. cached with noise), largest saving [13]; **relatedness-based**: related removal (distance), Shaw removal with distance/time/skill terms [1], a time-only Shaw variant [12], temporal Shaw on current service-begin times [10], and cluster removal [10]; **structural**: whole-route removal, contiguous sequence removal [13], and time-window segment removal (all customers served in a sampled time band); **history-based**: removal biased towards customers whose historically best-known insertion cost exceeds their current one [2]. Finally, **skill-scarcity removal** is problem-specific: it prefers customers whose skill set few couriers hold, freeing scarce shift capacity for reassignment. The number of removed customers q is sampled per call from a fraction range of n (base operators 5–15%, relatedness operators 5–20%), capped at $\min(100, 0.25n)$ to keep repair cost bounded on large instances.

3.4 Repair operators

Nine repair operators reinsert from a candidate set consisting of the removed customers plus a random sample of at most 40 additional unserved customers (this cap keeps repair cost independent of how many customers happen to be unserved): greedy best insertion; regret-2 over insertion positions [2]; regret-3 over *distinct couriers*, the meaningful regret when a scarce skill leaves only few compatible couriers; sequential cheapest insertion in profit order; noised greedy insertion; scarce-skill-first (insert customers with the fewest compatible couriers first); last-removed-first-inserted [13]; random-feasible-position insertion [13]; and Shaw insertion, which inserts customers most related to already-served ones. All repairs share the same incremental evaluation machinery (next section).

3.5 Efficient move evaluation

Pure Python forces the cost per move down. Each route carries a cache with arrival times, cumulative waiting and *forward time slacks* [4], so insertion feasibility at a position is $O(1)$ without route re-simulation; caches are rebuilt only for changed routes. On top sits a per-(customer, courier) move cache invalidated per route, so a repair pass rescans only the couriers whose routes changed. Candidate evaluation is penalized rather than rejected: time-window and shift violations cost 20 per time unit, skill violations a prohibitive 10^4 , so the SA search may traverse slightly infeasible states while the incumbent x^* is always feasible.

3.6 Acceptance, adaptivity and restarts

Acceptance is simulated annealing on the penalized score with $T_0 = 100$ (about one tenth of a typical customer profit accepted with probability e^{-1}) decaying exponentially to $T_{\min} = 10^{-3}$ *over the remaining wall-clock time*; a time-based schedule keeps the cooling profile identical across the two-orders-of-magnitude throughput range, which an iteration-based schedule cannot. Operator scores are 25 (new global best), 10 (better than current), 3 (accepted), 0 (rejected); weights are updated every $L = 100$ iterations ($L = 50$ for $n \geq 500$, where segments would otherwise outlast the run) with reaction factor 0.2 and are floored at 0.05. The utility divides the mean score by $1 + (\bar{t}/0.01\text{ s})^{0.5}$, so an operator that is twice as good but ten times slower loses weight—on $n = 1000$ this demotes the expensive regret repairs in favor of cheap greedy variants. After 500 consecutive rejections the current solution is replaced by a randomized greedy restart and the temperature is reheated to $0.75 T_0$. An optional or-opt polish hook exists but is disabled (it never paid for its runtime). The submitted solver uses a fixed seed and reserves 1 s of the budget for output, making every run deterministic and safely within the time limit.

4 Experimental Investigation of the Components

4.1 Operator ablation study

We removed 23 of the 24 operators individually (the time-only Shaw variant shares its implementation with Shaw removal and was not ablated separately) and reran the ALNS on three representative instances ($n = 100, 450, 1000$) with two seeds and 90 s per run; Table 1 reports the mean and worst change against the all-operator baseline. Three observations. First, *every* operator has a negative average impact

Table 1: Leave-one-out ablation: change in final profit when the operator is removed, relative to the all-operator baseline (3 instances $\times$ 2 seeds $\times$ 90s; negative = removal hurts). Baseline seed spread per instance: 0.1–1.8 %.

Operator	Type	avg %	worst %	Operator	Type	avg %	worst %
worst detour v2	destroy	−1.7	−4.7	noisy greedy ins.	repair	−0.3	−1.2
Shaw related	destroy	−1.1	−2.9	scarce-skill-first	repair	−0.3	−1.3
largest saving	destroy	−1.0	−2.9	Shaw insertion	repair	−0.3	−1.4
skill scarcity	destroy	−0.8	−1.6	regret-3 courier	repair	−0.2	−0.4
worst detour	destroy	−0.7	−1.6	random position	repair	−0.2	−1.3
cluster	destroy	−0.7	−2.0	random removal	destroy	−0.2	−0.7
time-window segment	destroy	−0.7	−2.1	related removal	destroy	−0.2	−2.0
temporal Shaw	destroy	−0.7	−1.6	last-removed-first	repair	−0.2	−1.0
greedy best ins.	repair	−0.6	−1.7	worst density	destroy	−0.2	−1.2
route removal	destroy	−0.5	−1.3	history-based	destroy	−0.1	−1.3
regret-2 insertion	repair	−0.4	−1.9				
sequence removal	destroy	−0.4	−1.5				
sequential profit	repair	−0.4	−1.4				

when removed—no operator is dead weight, which is itself a result of the adaptive weighting: cheap-but-weak operators are throttled automatically rather than harming the search. Second, the ranking is informative: the strongest contributors are the noised worst-detour removal (−1.7 % avg, −4.7 % worst) and Shaw relatedness removal—diversification through targeted, correlated removals matters most. The problem-specific skill-scarcity removal ranks fourth (−0.8 %), validating the design decision to encode skill scarcity in an operator. Third, magnitudes must be read against the noise floor: the baseline itself varies by 0.1–1.8 % between seeds, so the lower half of the table (operators around −0.2 %) is individually indistinguishable from noise. We keep this long tail nonetheless: removals are one-at-a-time, so joint removal effects are not additive; several “weak” operators have sharply negative worst cases (related removal: −2.0 %), i.e. they act as insurance on particular instance/seed combinations; and a superfluous operator costs only selection probability, which the weight mechanism reclaims within a few segments.

4.2 Parameter grid search

We tuned temperature scaling, destroy-size profile, repair candidate-pool cap, and stagnation limit in a grid of 16 configurations on three representative instances (two seeds, 30 s per run, common random numbers across configurations). The outcome was a *flat plateau*: the top configurations lay within 0.4 percentage points of average improvement, well inside seed noise. As a validation we reran the nominal winner (smaller destroy fractions, candidate pool 25, stagnation limit 700, start-profit-scaled T_0) on all 20 instances under the official 30 s protocol: 322,587 total profit vs. 323,172 for the defaults (−0.2 %, 9 instances better, 11 worse). A winner tuned on 3 instances does not generalize to the full benchmark, and no parameter direction shows a consistent signal. We therefore kept the default configuration of Section 3.6 and conclude that—within the examined ranges—the method is robust to its parameters; the components that do matter measurably are the operators themselves (Section 4.1) and the runtime-normalized adaptivity, not fine-tuned constants.

5 Experimental Investigation of the Overall Performance

5.1 Benchmark

Table 2 shows the submitted solver (`python3 skillvrp.py <instance> 30`) on all 20 instances. Every run finishes within the budget (29 s wall clock), every solution passes the provided checker, and the total collected profit is 323,172, on average 63.3 % of UB (range 55.3–73.1 %). The ALNS adds 26–62 % profit on top of the greedy start (mean 37 %). Since the solver seeds its RNG with a constant, these numbers are exactly reproducible; the seed sensitivity measured in Section 4.1 (0.1–1.8 % per instance) bounds the variation a different seed choice would cause. The spread in %UB says more about the bound than about the solutions: instances with only two or three couriers ($n = 100, 150$) or with high skill scarcity ($k = 2.0$: $n = 400, n = 950$) have many singleton-feasible customers competing for very little shift capacity, so UB is loose there; the best values (73–74 % at $n = 200$) occur where courier capacity is plentiful relative to n .

Table 2: Results on all 20 benchmark instances. UB = singleton upper bound; 30 s = submitted solver under the official budget (deterministic, checker verified); 600 s = long run; t_{last} = time of last improvement.

Instance	n	UB	greedy	30 s	%UB	600 s	%UB	t_{last} [s]
n50_v2_s3_k0.0	50	2717	1429	1798	66.2	1815	66.8	22
n75_v2_s4_k0.5	75	4255	1723	2793	65.6	2798	65.8	266
n100_v2_s5_k1.0	100	5160	2166	3050	59.1	3050	59.1	27
n125_v3_s6_k1.5	125	6866	2865	4315	62.8	4444	64.7	104
n150_v3_s8_k2.0	150	8259	3738	4882	59.1	4930	59.7	37
n200_v5_s3_k0.0	200	10236	5208	7480	73.1	7573	74.0	319
n250_v6_s4_k0.5	250	14034	7001	9948	70.9	10060	71.7	136
n300_v7_s5_k1.0	300	16729	8068	11364	67.9	11770	70.4	280
n350_v8_s6_k1.5	350	19050	8850	12709	66.7	12978	68.1	318
n400_v10_s8_k2.0	400	22335	10426	13167	59.0	13198	59.1	557
n450_v11_s3_k0.0	450	24478	10491	14270	58.3	14890	60.8	169
n500_v12_s4_k0.5	500	27820	13289	17847	64.2	18498	66.5	450
n600_v15_s5_k1.0	600	32477	15959	21456	66.1	22016	67.8	314
n700_v17_s6_k1.5	700	37852	18312	24137	63.8	25140	66.4	470
n750_v18_s8_k2.0	750	40329	18751	25266	62.7	26785	66.4	551
n800_v20_s3_k0.0	800	43128	19733	26022	60.3	27131	62.9	485
n850_v21_s4_k0.5	850	46190	21134	29336	63.5	31796	68.8	577
n900_v22_s5_k1.0	900	49703	22274	28264	56.9	30243	60.8	587
n950_v23_s6_k1.5	950	52946	22452	29254	55.3	31545	59.6	447
n1000_v25_s8_k2.0	1000	54664	27323	35814	65.5	38508	70.4	591
mean / total			46.4%UB	323172	63.3		65.5	

5.2 Long-run results

To measure what the 30 s budget costs, we reran all instances for 600 s with convergence tracking (Fig. 2; the configuration differs from the submitted defaults only in settings shown to be performance-neutral in Section 4.2). Three findings. First, the 30 s budget is efficient: on average the 30 s point already reaches 97.8 % of the 600 s result (minimum 95.4 %), because the time-based cooling schedule compresses the same annealing profile into whatever budget is given. Second, instance size determines convergence: instances up to $n \approx 350$ converge within one to five minutes (t_{last} 22–320 s), while all instances with $n \geq 750$ still improve in the final minute ($t_{\text{last}} \geq 447$ s)—on large instances the search is budget-bound, not converged, and more time or a faster language would translate directly into profit. Third, the extra 570 s buy on average 2.2 percentage points of UB (63.3 % $\rightarrow$ 65.5 %), with the largest gains exactly on the large, budget-bound instances ($n = 850$: +5.3 points, $n = 1000$: +4.9). All 600 s solutions are feasible and checker-verified.

6 Conclusion

We presented an ALNS for the prize-collecting Skill VRP that serves an average of 63.3 % of the singleton upper bound across all 20 benchmark instances within the official 30 s budget, with feasibility verified by the provided checker on every instance. The main lessons match the development path: (i) fix the yardstick first—the cheap singleton bound made every later design decision measurable; (ii) throughput before operator richness— $O(1)$ insertion evaluation via time slacks and per-route caches was worth more than any single operator; (iii) measure instead of argue—the ablation showed that every operator contributes on average and that the problem-specific skill-scarcity removal earns its place, while the parameter study showed a flat plateau that made further tuning pointless. Future work: slack-induced string removals [14] as a more disruptive destroy step for the budget-bound large instances, insertion scoring that accounts for consumed shift time rather than pure travel delta, and a set-covering post-optimization over the pool of routes collected during the search.

References

- [1] P. Shaw. Using constraint programming and local search methods to solve vehicle routing problems. *CP 1998*, LNCS 1520, 417–431, 1998.

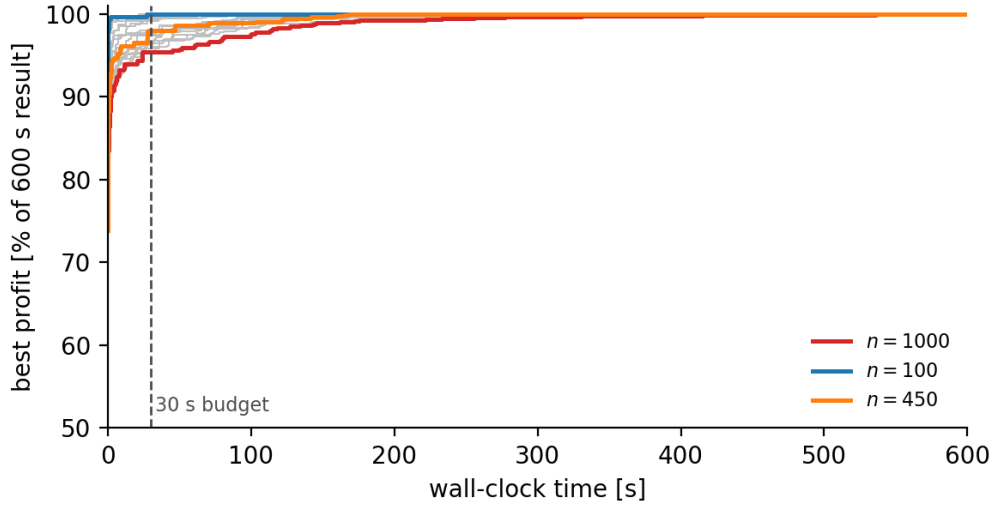


Figure 2: Best-solution convergence over 600s, normalized to the final value; gray = all 20 instances, highlighted = $n = 100/450/1000$. The dashed line marks the official 30s budget, which captures on average 97.8 % of the 600s quality.

- [2] S. Ropke, D. Pisinger. An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. *Transportation Science* 40(4):455–472, 2006.
- [3] D. Pisinger, S. Ropke. A general heuristic for vehicle routing problems. *Computers & Operations Research* 34(8):2403–2435, 2007.
- [4] M. W. P. Savelsbergh. The vehicle routing problem with time windows: minimizing route duration. *ORSA Journal on Computing* 4(2):146–154, 1992.
- [5] P. Vansteenwegen, W. Souffriau, D. Van Oudheusden. The orienteering problem: a survey. *European Journal of Operational Research* 209(1):1–10, 2011.
- [6] A. Gunawan, H. C. Lau, P. Vansteenwegen. Orienteering problem: a survey of recent variants, solution approaches and applications. *European Journal of Operational Research* 255(2):315–332, 2016.
- [7] N. Labadie, R. Mansini, J. Melechovský, R. Wolfler Calvo. The team orienteering problem with time windows: an LP-based granular variable neighborhood search. *European Journal of Operational Research* 220(1):15–27, 2012.
- [8] Q. Hu, A. Lim. An iterative three-component heuristic for the team orienteering problem with time windows. *European Journal of Operational Research* 232(2):276–286, 2014.
- [9] P. Cappanera, L. Gómez-Real, M. G. Scutellà. The skill vehicle routing problem. *Network Optimization (INOC 2011)*, LNCS 6701, 354–364, 2011.
- [10] A. A. Kovacs, S. N. Parragh, K. F. Doerner, R. F. Hartl. Adaptive large neighborhood search for service technician routing and scheduling problems. *Journal of Scheduling* 15(5):579–600, 2012.
- [11] M. Cissé, S. Yalçındağ, Y. Kergosien, E. Şahin, C. Lenté, A. Matta. OR problems related to home health care: a review of relevant routing and scheduling problems. *Operations Research for Health Care* 13–14:1–22, 2017.
- [12] E. Demir, T. Bektaş, G. Laporte. An adaptive large neighborhood search heuristic for the pollution-routing problem. *European Journal of Operational Research* 223(2):346–359, 2012.
- [13] F. Hammami, M. Rekik, L. C. Coelho. A hybrid adaptive large neighborhood search heuristic for the team orienteering problem. *Computers & Operations Research* 123:105034, 2020.

- [14] J. Christiaens, G. Vanden Berghe. Slack induction by string removals for vehicle routing problems. *Transportation Science* 54(2):417–433, 2020.
- [15] T. Vidal. Hybrid genetic search for the CVRP: open-source implementation and SWAP* neighborhood. *Computers & Operations Research* 140:105643, 2022.